

APOSTILA DE COMPILADORES

Aula 10

Análise LL(1) e construção do parser preditivo

Ciência da Computação — 2026/2

Professor: Alex Torquato Souza Carneiro

Projeto integrador: linguagem MINIC

Material de apoio para a aula sobre gramáticas LL(1), tabelas preditivas, implementação de parser não recursivo e recuperação de erros.

1. Visão geral da aula

Na Aula 9 estudamos a análise sintática descendente, a eliminação da recursão à esquerda, a fatoração à esquerda e os conjuntos FIRST e FOLLOW. Nesta aula, esses elementos são reunidos em um método determinístico: o analisador preditivo LL(1).

O objetivo é transformar uma gramática adequada em uma tabela de análise e, em seguida, utilizar essa tabela para reconhecer programas da MINIC, construir a AST e emitir diagnósticos sintáticos úteis.

Conteúdos previstos no plano de aulas: definição de LL(1); construção de tabelas preditivas; uso de FIRST e FOLLOW; conflitos; implementação de parser preditivo; recuperação de erros; sincronização por tokens.

Ao final, você deverá ser capaz de explicar por que uma gramática é ou não LL(1), preencher a tabela $M[A,a]$, simular a pilha do parser e projetar uma estratégia básica de recuperação de erros.

2. O que significa LL(1)?

A sigla LL(1) descreve um analisador que lê a entrada da esquerda para a direita (primeiro L), produz uma derivação mais à esquerda (segundo L) e toma decisões usando um símbolo de lookahead (1).

Em cada passo, o parser precisa escolher uma única produção para expandir o não-terminal que está no topo da pilha. A escolha deve ser determinada pelo próximo token, sem retrocesso.

Consequência prática: uma gramática com recursão à esquerda ou com alternativas que começam pelo mesmo padrão normalmente precisa ser transformada antes da construção da tabela.

Para uma produção $A \rightarrow \alpha \mid \beta$, as alternativas devem ser distinguíveis pelo lookahead. Em termos de conjuntos, $FIRST(\alpha)$ e $FIRST(\beta)$ não podem se sobrepor. Se uma alternativa puder gerar ϵ , os símbolos de $FOLLOW(A)$ também precisam ser considerados.

3. FIRST e FOLLOW: a base da previsão

O conjunto $FIRST(\alpha)$ contém os terminais que podem iniciar alguma cadeia derivada de α . Se α puder derivar a cadeia vazia ϵ , então ϵ também pertence a $FIRST(\alpha)$.

O conjunto $FOLLOW(A)$ contém os terminais que podem aparecer imediatamente depois do não-terminal A em alguma forma sentencial. Para o símbolo inicial, o marcador de fim de entrada $\$$ pertence a $FOLLOW(S)$.

Uso no parser: FIRST indica quando uma produção pode começar; FOLLOW indica quando uma produção- ϵ pode ser aplicada e também ajuda a definir tokens de sincronização durante a recuperação de erros.

Não-terminal	FIRST	FOLLOW
E	{ (, id }	{), \$ }
E'	{ +, ϵ }	{), \$ }
T	{ (, id }	{ +,), \$ }
T'	{ *, ϵ }	{ +,), \$ }
F	{ (, id }	{ *, +,), \$ }

Tabela 1 — Exemplo de FIRST e FOLLOW para a gramática de expressões (adaptado de Aho, Sethi e Ullman, Cap. 4).

4. Gramática de expressões para o exemplo

Para evitar ambiguidade e recursão à esquerda, utilizaremos a forma clássica da gramática de expressões:

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → ( E ) | id
```

Essa gramática representa a precedência usual: a multiplicação é tratada em T e a adição em E. As produções com ϵ permitem encerrar uma sequência de operadores quando o lookahead pertence a FOLLOW do não-terminal correspondente.

Exemplo: para a entrada `id + id * id`, o parser expande E com $E \rightarrow T E'$, reconhece o primeiro `id`, escolhe $E' \rightarrow + T E'$ ao encontrar `+`, e depois escolhe $T' \rightarrow * F T'$ ao encontrar `*`.

Figura de apoio do livro

corpo começa com F . Como F não deriva ϵ , o $\text{FIRST}(T)$ deve ser igual ao $\text{FIRST}(F)$. O mesmo argumento se aplica ao $\text{FIRST}(E)$.

2. $\text{FIRST}(E') = \{+, \epsilon\}$. O motivo é que uma das duas produções para E' tem um corpo que começa com o terminal $+$, e o corpo da outra é ϵ . Sempre que um não-terminal deriva ϵ , incluímos ϵ em FIRST para esse não-terminal.
3. $\text{FIRST}(T') = \{*, \epsilon\}$. O raciocínio é semelhante àquele usado no $\text{FIRST}(E')$.
4. $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$\}$. Como E é o símbolo inicial da gramática, $\text{FOLLOW}(E)$ deve incluir $\$$. O corpo (E) da produção- F explica por que o parêntese direito está em $\text{FOLLOW}(E)$. Para E' , observe que esse não-terminal aparece apenas nas extremidades dos corpos das produções- E . Assim, $\text{FOLLOW}(E')$ deve ser o mesmo que $\text{FOLLOW}(E)$.
5. $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{+, \epsilon, \$\}$. Observe que T aparece do lado direito das produções- E apenas seguido por E' . Assim, tudo exceto ϵ que está em $\text{FIRST}(E')$ deve ser incluído em $\text{FOLLOW}(T)$; isso explica o símbolo $+$. Contudo, como $\text{FIRST}(E')$ contém ϵ (ou seja, $E' \xRightarrow{*} \epsilon$), e E' é a cadeia inteira seguindo T nos corpos das produções- E , tudo em $\text{FOLLOW}(E)$ também deve ser incluído em $\text{FOLLOW}(T)$. Isso explica os símbolos $\$$ e o parêntese direito. Quanto a T' , por aparecer apenas nas extremidades das produções- T , é necessário fazer o $\text{FOLLOW}(T') = \text{FOLLOW}(T)$.
6. $\text{FOLLOW}(F) = \{+, *, \epsilon, \$\}$. O raciocínio é semelhante àquele para T no ponto (5).

4.4.3 GRAMÁTICAS LL(1)

Os analisadores sintáticos preditivos, ou seja, os analisadores de descida recursiva que não precisam de retrocesso, podem ser construídos para uma classe de gramáticas chamada LL(1). O primeiro “L” em LL(1) significa que a cadeia de entrada é escandida da esquerda para a direita ($L = \text{Left-to-right}$); o segundo “L” representa uma derivação mais à esquerda ($L = \text{Leftmost}$); e o “1” pelo uso de um símbolo à frente na entrada utilizado em cada passo para tomar as decisões quanto à ação de análise.

Diagramas de transição para analisadores sintáticos preditivos

Os diagramas de transição são úteis para visualizar analisadores preditivos. Por exemplo, os diagramas de transição para os não-terminais E e E' da gramática (4.28) são ilustrados na Figura 4.16(a). Para construir o diagrama de transição de uma gramática, primeiro elimine a recursão à esquerda e depois fatore a gramática à esquerda. Então, para cada não-terminal A :

1. Crie os estados inicial e final (retorno).
2. Para cada produção- $A \rightarrow X_1 X_2 \dots X_k$, crie um caminho do estado inicial para o estado final, com as arestas rotuladas com $X_1 X_2 \dots X_k$. Se $A \rightarrow \epsilon$, o caminho é uma aresta rotulada com ϵ .

Os diagramas de transição para analisadores preditivos diferem daqueles para os analisadores léxicos. Os analisadores sintáticos possuem um diagrama para cada não-terminal. Os rótulos das arestas podem ser tokens ou não-terminais. Uma transição sob um token (terminal) significa que efetuaremos essa transição se esse token for o próximo símbolo de entrada. Uma transição sob um não-terminal A representa a ativação do procedimento A .

Com uma gramática LL(1), a ambigüidade de seguir ou não uma aresta ϵ pode ser resolvida tornando as transições- ϵ a escolha *default*.

Os diagramas de transição podem ser simplificados, desde que a seqüência de símbolos da gramática ao longo dos caminhos seja preservada. Também podemos substituir o diagrama de um não-terminal A por uma aresta rotulada com A . Os diagramas da Figura 4.16(a) e (b) são equivalentes: se traçarmos caminhos de E para um estado de aceitação com E' substituído, então, nos dois conjuntos de diagramas, os símbolos da gramática ao longo dos caminhos compõem cadeias da forma $T + T + \dots + T$. O diagrama em (b) pode ser obtido de (a) pelas transformações semelhantes àquelas na Seção 2.5.4, onde usamos a remoção da recursão de cauda e a substituição dos corpos dos procedimentos para otimizar o procedimento para um não-terminal.

A classe de gramáticas LL(1) é rica o suficiente para reconhecer a maioria das construções presentes nas linguagens de programação, mas escrever uma gramática adequada para a linguagem fonte não é uma tarefa simples. Por exemplo, nenhuma gramática com recursão à esquerda ou ambígua pode ser LL(1).

Uma gramática G é LL(1) se e somente se, sempre que $A \rightarrow \alpha/\beta$ forem duas produções distintas de G , as seguintes condições forem verdadeiras:

1. Para um terminal a , tanto α quanto β não derivam cadeias começando com a .
2. No máximo um dos dois, α ou β , pode derivar a cadeia vazia.

Figura 1 — Trecho do Capítulo 4 com a discussão de FIRST, FOLLOW e gramáticas LL(1). Fonte: Aho, Sethi e Ullman, Compiladores: princípios, técnicas e ferramentas, material fornecido pelo docente.

5. Construção da tabela preditiva $M[A,a]$

A tabela M possui uma linha para cada não-terminal e uma coluna para cada terminal, incluindo o marcador $\$$. Para cada produção $A \rightarrow \alpha$:

- insira $A \rightarrow \alpha$ em $M[A,a]$ para cada terminal $a \in \text{FIRST}(\alpha)$;
- se $\epsilon \in \text{FIRST}(\alpha)$, insira $A \rightarrow \alpha$ em $M[A,b]$ para cada $b \in \text{FOLLOW}(A)$;
- as entradas que permanecem vazias representam situações de erro.

Em uma gramática LL(1), cada célula possui no máximo uma produção. Duas produções na mesma célula revelam um conflito: a decisão não pode ser tomada com apenas um token de lookahead.

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Tabela 2 — Esquema simplificado da tabela M para a gramática de expressões. Células vazias indicam erro.

6. Parser preditivo com pilha explícita

O parser não recursivo mantém três componentes principais: buffer de entrada, pilha e tabela M . Inicialmente, a pilha contém $\$$ e o símbolo inicial E ; a entrada termina em $\$$.

A cada iteração, observe X , o topo da pilha, e a , o token corrente:

- se $X = a = \$$, aceite a entrada;
- se X é terminal e $X = a$, desempilhe X e avance o ponteiro;
- se X é terminal e $X \neq a$, emita erro de casamento;
- se X é não-terminal, consulte $M[X,a]$;
- se a célula contém $X \rightarrow Y_1 Y_2 \dots Y_k$, desempilhe X e empilhe $Y_k \dots Y_1$, deixando Y_1 no topo.

A saída do parser pode registrar as produções aplicadas. Em uma implementação real, a mesma decisão também cria nós da AST, associa tokens aos nós e preserva posições de linha e coluna.

Pseudocódigo do analisador

```

inicialize pilha com [ $\$, S$ ]
enquanto topo  $\neq \$$ :
   $X \leftarrow$  topo da pilha;  $a \leftarrow$  token corrente
  se  $X = a$ : desempilhe  $X$ ; avance a entrada
  senão se  $X$  é terminal: erro de casamento
  senão se  $M[X,a] = \text{erro}$ : recuperação
  senão aplique  $X \rightarrow Y_1 \dots Y_k$ ; substitua  $X$  por  $Y_k \dots Y_1$ 
  aceite se  $a = \$$ 

```

Modelo visual do livro

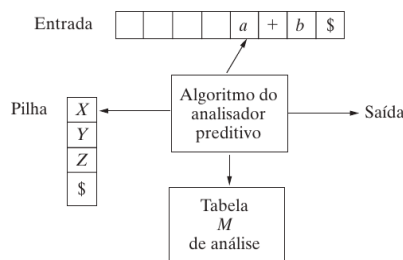


FIGURA 4.19 Modelo de um analisador sintático preditivo dirigido por tabela.

O analisador é dirigido por um programa que considera X , o símbolo no topo da pilha, e a , o símbolo corrente da entrada. Se X é um não-terminal, o analisador escolhe uma produção- X consultando a entrada $M[X, a]$ da tabela M de análise. (Códigos adicionais poderiam ser executados neste ponto, por exemplo, o código para construir um nó em uma árvore de derivação.) Por outro lado, ele tenta fazer um casamento entre o terminal X no topo da pilha e o símbolo corrente a da entrada.

O comportamento do analisador pode ser descrito em termos de suas *configurações*, que mostram em cada passo o conteúdo da pilha e a entrada restante. O próximo algoritmo descreve como essas configurações são manipuladas.

ALGORITMO 4.34: Reconhecimento preditivo dirigido por tabela.

ENTRADA: Uma cadeia w de entrada e uma tabela M de análise para a gramática G .

SAÍDA: A derivação mais à esquerda de w , se w estiver em $L(G)$; caso contrário, uma indicação de erro.

MÉTODO: Inicialmente, o analisador está em uma configuração com $w\$$ no buffer de entrada e o símbolo inicial S de G no topo da pilha, acima de S . O programa da Figura 4.20 usa a tabela M de análise para produzir um analisador preditivo para a entrada.

```
define  $ip$  para que aponte para o primeiro símbolo de  $w$ ;
define  $X$  para ser o símbolo no topo da pilha;
while (  $X \neq \$$  ) { /* pilha não está vazia */
  if (  $X$  é  $a$  ) desempilha e avança  $ip$ ;
  else if (  $X$  é um terminal ) error();
  else if (  $M[X, a]$  é uma entrada de erro ) error();
  else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  ) {
    imprime a produção  $X \rightarrow Y_1 Y_2 \dots Y_k$ ;
    desempilha  $X$ ;
    empilha  $Y_k, Y_{k-1}, \dots, Y_1$  na pilha, com  $Y_1$  no topo;
  }
  define  $X$  como o símbolo no topo da pilha;
}
```

FIGURA 4.20 Algoritmo do analisador preditivo.

EXEMPLO 4.35: Considere a gramática (4.28); já vimos sua tabela de análise na Figura 4.17. Para a entrada **id + id * id**, o analisador preditivo não-recursivo do Algoritmo 4.34 efetua a sequência de movimentos ilustrados na Figura 4.21. Esses movimentos correspondem a uma derivação mais à esquerda (veja a derivação completa na Figura 4.12).

$$E \xRightarrow{im} TE' \xRightarrow{im} FT'E' \xRightarrow{im} idT'E' \xRightarrow{im} idE' \xRightarrow{im} id + TE' \xRightarrow{im} \dots$$

Observe que as formas sentenciais nessa derivação correspondem à entrada que já foi casada (na coluna CASAMENTO) seguida pelo conteúdo da pilha. A entrada casada aparece apenas para realçar a correspondência. Pelo mesmo motivo, o topo da pilha está à esquerda; quando considerarmos a análise ascendente, será mais natural mostrar o topo da pilha à direita. O apontador de entrada aponta para o símbolo mais à esquerda da cadeia na coluna ENTRADA.

Figura 2 — Modelo de um analisador sintático preditivo dirigido por tabela e algoritmo associado. Fonte: Aho, Sethi e Ullman, Cap. 4.

7. Simulação: id + id * id

A simulação abaixo mostra o comportamento essencial do parser. O topo da pilha aparece à esquerda para destacar a próxima decisão.

Pilha	Entrada	Ação
E \$	id + id * id \$	$M[E, id] = E \rightarrow TE'$
T E' \$	id + id * id \$	$M[T, id] = T \rightarrow FT'$
F T' E' \$	id + id * id \$	$M[F, id] = F \rightarrow id$
id T' E' \$	id + id * id \$	casa id
T' E' \$	+ id * id \$	$M[T', +] = T' \rightarrow \epsilon$
E' \$	+ id * id \$	$M[E', +] = E' \rightarrow +TE'$
+ T E' \$	+ id * id \$	casa +
T E' \$	id * id \$	repete T
...	...	continua até \$

Tabela 3 — Primeiros passos de uma simulação de análise preditiva.

Durante a construção da AST, uma produção como $E \rightarrow TE'$ cria ou organiza o nó correspondente à expressão; tokens casados fornecem folhas, lexemas e atributos. Produções- ϵ normalmente não geram nós sintáticos visíveis.

8. Conflitos e diagnóstico da gramática

Um conflito aparece quando duas produções são inseridas na mesma célula $M[A, a]$. Isso pode ocorrer por recursão à esquerda, ambiguidade ou fatoração insuficiente.

Exemplo conceitual: se $A \rightarrow \alpha \mid \beta$ e $FIRST(\alpha) \cap FIRST(\beta) \neq \emptyset$, o token corrente não distingue as alternativas. Se uma alternativa deriva ϵ e $FIRST$ da outra intersecta $FOLLOW(A)$, também há conflito.

- elimine a recursão à esquerda;
- aplique fatoração à esquerda;
- reavalie $FIRST$ e $FOLLOW$;
- se a linguagem não admitir uma gramática LL(1) equivalente, considere outro método de parsing, como LL(k), LR ou uma estratégia específica para a construção ambígua.

9. Recuperação de erros por modo pânico

O erro é detectado quando um terminal no topo da pilha não coincide com o token corrente ou quando $M[A, a]$ está vazia. No modo pânico, o parser descarta tokens até encontrar um token de sincronização que permita retomar a análise.

Uma heurística inicial é usar $FOLLOW(A)$ como conjunto de sincronização para o não-terminal A. Também podem ser acrescentados tokens de $FIRST(A)$, delimitadores de comandos e palavras-chave que iniciam construções de nível superior.

Exemplo: em uma expressão dentro de um comando, ponto e vírgula, parêntese direito e palavras-chave que iniciam o próximo comando podem ser bons pontos de retomada. A mensagem

deve indicar o token encontrado, o contexto esperado e a posição no programa.

Recuperação de erros no livro

cap04_AHOg.qxd 26.10.07 14:34 Page 147



EXEMPLO 4.36: Usar os símbolos de FIRST e FOLLOW como tokens de sincronismo funciona razoavelmente bem quando as expressões são analisadas de acordo com uma gramática do tipo daquela proposta em (4.28). A tabela de análise para essa gramática, ilustrada na Figura 4.17, é repetida na Figura 4.22, com “synch” indicando os tokens de sincronização obtidos do conjunto FOLLOW do não-terminal em questão. Os conjuntos do FOLLOW para os não-terminais são obtidos do Exemplo 4.30.

A tabela da Figura 4.22 deve ser usada da forma a seguir. Se o analisador sintático olhar a entrada $M[A, a]$ e descobrir que ela está em branco, então o símbolo a da entrada é ignorado. Se a entrada é “synch”, então o não-terminal do topo da pilha é desempilhado na tentativa de retomar a análise. Se um token no topo da pilha não casar com o símbolo da entrada, então desempilhamos o token da pilha, conforme mencionamos anteriormente.

NÃO TERMINAL	SÍMBOLO DE ENTRADA					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E \rightarrow + TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$	synch	synch	$F \rightarrow (E)$	synch	synch

FIGURA 4.22 Tokens de sincronização incluídos à tabela de análise da Figura 4.17.

Para a entrada errônea $) id * + id$, o analisador sintático e a abordagem de recuperação de erro da Figura 4.22 se comportam como na Figura 4.23.

PILHA	ENTRADA	COMENTÁRIO
$E \$$	$) id * + id \$$	erro, pula)
$E \$$	$id * + id \$$	id está em $FIRST(E)$
$TE' \$$	$id * + id \$$	
$FT'E' \$$	$id * + id \$$	
$id TE' \$$	$id * + id \$$	
$TE' \$$	$* + id \$$	
$* FT'E' \$$	$* + id \$$	
$FT'E' \$$	$+ id \$$	erro, $M[F, +] = \text{synch}$
$TE' \$$	$+ id \$$	F foi desempilhado
$E' \$$	$+ id \$$	
$+ TE' \$$	$+ id \$$	
$TE' \$$	$id \$$	
$FT'E' \$$	$id \$$	
$id TE' \$$	$id \$$	
$TE' \$$	$\$$	
$E' \$$	$\$$	
$\$$	$\$$	

FIGURA 4.23 Movimentos da análise e recuperação de erro efetuados por um analisador sintático preditivo.

A discussão anterior sobre recuperação de erro no modo pânico não resolve a importante questão das mensagens de erro. O projetista do compilador precisa disponibilizar mensagens de erro precisas, que não apenas descrevam o erro, mas chamem a atenção para o local em que o erro foi descoberto.

Recuperação em nível de frase

A recuperação de erros em nível de frase é implementada preenchendo-se as entradas em branco da tabela do analisador preditivo com apontadores para rotinas de erro. Essas rotinas podem substituir, inserir ou excluir símbolos da entrada e emitir mensagens de erro apropriadas. Elas também podem remover símbolos da pilha. A alteração dos símbolos de pilha ou o empilhamento de novos símbolos é questionável por vários motivos. Primeiro, os passos executados pelo analisador poderiam não corresponder à derivação de nenhuma construção da linguagem. Segundo, temos de garantir que não haverá possibilidade de um *loop* infinito. Uma ótima maneira de se proteger contra esses *loops* é verificar se alguma ação de recuperação consome um símbolo de entrada (ou se a pilha se tornou menor quando o fim da entrada foi alcançado).

Figura 3 — Tabela com tokens de sincronização e exemplo de movimentos durante a recuperação. Fonte: Aho, Sethi e Ullman, Cap. 4.

10. Aplicação no projeto MINIC

O parser da MINIC deve receber a sequência de tokens produzida pelo scanner e devolver uma AST ou uma lista estruturada de erros. A tabela preditiva é construída a partir da gramática da linguagem, já transformada para ser compatível com LL(1).

Integração scanner-parser: o scanner informa tipo, lexema, linha e coluna; o parser consulta apenas o tipo do token para decidir a produção e utiliza o lexema para compor identificadores, literais e mensagens.

- representar cada produção da gramática em uma estrutura acessível pelo parser;
- manter uma pilha de símbolos e, quando necessário, uma pilha de nós da AST;
- registrar erros sem interromper o primeiro diagnóstico;
- sincronizar em `;`, `}`, `)`, palavras-chave e tokens de início de comandos;
- testar entradas válidas, inválidas e casos de fronteira.

11. Atividade prática

Parte A — tabela: para a gramática de expressões, calcule FIRST e FOLLOW e preencha $M[A,a]$. Identifique todas as células de erro.

Parte B — simulação: execute manualmente o parser para `id + id * id`, registrando pilha, entrada e ação a cada passo.

Parte C — implementação: implemente o laço do parser preditivo e conecte-o ao scanner da MINIC. Cada aplicação de produção deve ser registrada no modo de depuração.

Parte D — erros: teste entradas com operador ausente, parêntese não fechado, comando sem ponto e vírgula e token inesperado. Para cada caso, registre o token de sincronização e a mensagem exibida.

12. Checklist de aprendizagem

- Sei explicar cada componente da sigla LL(1).
- Consigo calcular FIRST de símbolos e cadeias.
- Consigo calcular FOLLOW de não-terminais.
- Sei preencher uma tabela $M[A,a]$.
- Consigo distinguir casamento, expansão, aceitação e erro.
- Consigo explicar como a pilha simula uma derivação mais à esquerda.
- Sei definir tokens de sincronização para recuperação em modo pânico.
- Consigo integrar o parser à construção da AST da MINIC.

Síntese: LL(1) transforma a escolha de produções em uma decisão tabular baseada em um único token. A qualidade do parser depende tanto da gramática quanto da implementação da pilha, da construção da AST e das mensagens de erro.

Referência principal: AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. Capítulo 4, seção 4.4 — análise sintática descendente, especialmente as subseções sobre FIRST e FOLLOW, gramáticas LL(1), analisador preditivo sem recursão e recuperação de erros. Figuras reproduzidas como material de apoio a partir do livro fornecido para esta atividade.